

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

ОТЧЕТ

по лабораторной работе N 1

**Создание серверного приложения на Node.js и
Express.js**

Тема проекта: платформа для обмена игровыми скриншотами и видео

Выполнил: студент группы _____

Проверил:

Минск 2026

1 Цель работы

Приобретение практических навыков создания серверного приложения на платформе Node.js с использованием Express.js, изучение маршрутизации, обработки HTTP-запросов GET, POST, PUT и DELETE, формирования JSON-ответов и обработки ошибок.

2 Описание проекта

Разработан REST API для платформы обмена игровыми скриншотами и видео. Основной ресурс назван posts. Публикация содержит идентификатор, заголовок, название игры, тип медиа, ссылку, описание и комментарии. Данные временно хранятся в массиве без подключения базы данных.

3 Структура проекта

Для возможности расширения в следующих лабораторных работах код разделен на маршруты, контроллер и модель. Дополнительные технологии не используются.

```
Lab1/
|-- routes/
|   |-- postRoutes.js      # адреса маршрутов
|-- controllers/
|   |-- postController.js  # CRUD и проверка данных
|-- models/
|   |-- postModel.js       # временный массив публикаций
|-- server.js              # настройка и запуск Express
|-- requests.http
|-- Game Media API.postman_collection.json
|-- package.json
`-- output/pdf/report_lab1.pdf
```

4 Маршруты API

Метод	Маршрут	Назначение	Статусы
GET	/posts	Получить список публикаций	200, 400
GET	/posts/:id	Получить публикацию по ID	200, 404
POST	/posts	Создать публикацию	201, 400
PUT	/posts/:id	Полностью обновить публикацию	200, 400, 404
DELETE	/posts/:id	Удалить публикацию	204, 404

Сервер запускается на порту 3001, так как порт 3000 занят локальной службой. Для JSON используется Content-Type: application/json. Ошибки возвращаются в виде { error: "message" }.

5 ОСНОВНЫЕ ЛИСТИНГИ

server.js

```
1 const express = require('express');
2 const postRoutes = require('./routes/postRoutes');
3
4 const app = express();
5 const PORT = 3001;
6
7 app.use(express.json());
8
9 app.get('/', (req, res) => {
10   res.json({ message: 'Game Media API is running' });
11 });
12
13 app.use('/posts', postRoutes);
14
15 app.use((req, res) => {
16   res.status(404).json({ error: 'Route not found' });
17 });
18
19 app.use((err, req, res, next) => {
20   console.error(err);
21
22   if (err instanceof SyntaxError && err.status === 400) {
23     return res.status(400).json({ error: 'Invalid JSON' });
24   }
25
26   return res.status(500).json({ error: 'Internal server error' });
27 });
28
29 app.listen(PORT, () => {
30   console.log(`Server is running at http://localhost:${PORT}`);
31 });
```

routes/postRoutes.js

```
1 const express = require('express');
2 const {
3   getPosts,
4   getPostById,
5   createPost,
6   updatePost,
7   deletePost,
8 } = require('../controllers/postController');
9
10 const router = express.Router();
11
12 router.get('/', getPosts);
13 router.get('/:id', getPostById);
14 router.post('/', createPost);
15 router.put('/:id', updatePost);
16 router.delete('/:id', deletePost);
17
18 module.exports = router;
```

models/postModel.js

```
1 const posts = [
2   {
3     id: 1,
4     title: 'Sunset in Night City',
5     game: 'Cyberpunk 2077',
6     mediaType: 'screenshot',
7     mediaUrl: 'https://example.com/night-city.jpg',
8     description: 'A screenshot taken during an evening walk.',
9     comments: ['Great lighting!', 'This city looks amazing.'],
10   },
11   {
12     id: 2,
13     title: 'Winning goal',
14     game: 'Rocket League',
15     mediaType: 'video',
16     mediaUrl: 'https://example.com/winning-goal.mp4',
17     description: 'The final goal of the match.',
18     comments: ['Nice shot!'],
19   },
20 ];
21
```

```
22 let nextId = 3;
23
24 function getNextId() {
25   return nextId++;
26 }
27
28 module.exports = { posts, getNextId };
```

controllers/postController.js

```
1  const { posts, getNextId } = require('../models/postModel');
2
3  function validatePost(post) {
4    if (!post || typeof post !== 'object' || Array.isArray(post)) {
5      return 'Request body must be a JSON object';
6    }
7
8    const requiredStringFields = ['title', 'game', 'mediaType', 'mediaUrl', 'description'];
9
10   for (const field of requiredStringFields) {
11     if (typeof post[field] !== 'string' || post[field].trim() === '') {
12       return `Field "${field}" must be a non-empty string`;
13     }
14   }
15
16   if (!['screenshot', 'video'].includes(post.mediaType)) {
17     return 'Field "mediaType" must be "screenshot" or "video"';
18   }
19
20   if (
21     post.comments !== undefined &&
22     (!Array.isArray(post.comments) ||
23      !post.comments.every((comment) => typeof comment === 'string'))
24   ) {
25     return 'Field "comments" must be an array of strings';
26   }
27
28   return null;
29 }
30
31 function getPosts(req, res) {
32   const { mediaType } = req.query;
33
34   if (mediaType && !['screenshot', 'video'].includes(mediaType)) {
35     return res.status(400).json({
36       error: 'Query parameter "mediaType" must be "screenshot" or "video"',
37     });
38   }
39
40   const result = mediaType
41     ? posts.filter((post) => post.mediaType === mediaType)
42     : posts;
```

controllers/postController.js (продолжение)

```
43
44   return res.json(result);
45 }
46
47 function getPostById(req, res) {
48   const id = Number(req.params.id);
49   const post = posts.find((item) => item.id === id);
50
51   if (!post) {
52     return res.status(404).json({ error: 'Post not found' });
53   }
54
55   return res.json(post);
56 }
57
58 function createPost(req, res) {
59   const validationError = validatePost(req.body);
60
61   if (validationError) {
62     return res.status(400).json({ error: validationError });
63   }
64
65   const newPost = {
66     id: getNextId(),
67     title: req.body.title.trim(),
68     game: req.body.game.trim(),
69     mediaType: req.body.mediaType,
70     mediaUrl: req.body.mediaUrl.trim(),
71     description: req.body.description.trim(),
72     comments: req.body.comments || [],
73   };
74
75   posts.push(newPost);
76   return res.status(201).json(newPost);
77 }
78
79 function updatePost(req, res) {
80   const id = Number(req.params.id);
81   const postIndex = posts.findIndex((item) => item.id === id);
82
83   if (postIndex === -1) {
84     return res.status(404).json({ error: 'Post not found' });
```

controllers/postController.js (продолжение)

```
85   }
86
87   const validationError = validatePost(req.body);
88
89   if (validationError) {
90     return res.status(400).json({ error: validationError });
91   }
92
93   const updatedPost = {
94     id,
95     title: req.body.title.trim(),
96     game: req.body.game.trim(),
97     mediaType: req.body.mediaType,
98     mediaUrl: req.body.mediaUrl.trim(),
99     description: req.body.description.trim(),
100    comments: req.body.comments || [],
101  };
102
103  posts[postIndex] = updatedPost;
104  return res.json(updatedPost);
105 }
106
107 function deletePost(req, res) {
108   const id = Number(req.params.id);
109   const postIndex = posts.findIndex((item) => item.id === id);
110
111   if (postIndex === -1) {
112     return res.status(404).json({ error: 'Post not found' });
113   }
114
115   posts.splice(postIndex, 1);
116   return res.status(204).send();
117 }
118
119 module.exports = {
120   getPosts,
121   getPostById,
122   createPost,
123   updatePost,
124   deletePost,
125 };
```

6 Тестирование API

Запросы выполнены последовательно по адресу `http://localhost:3001`. Коллекция Postman содержит те же запросы в демонстрационном порядке.



Рисунок 1 - Получение списка публикаций

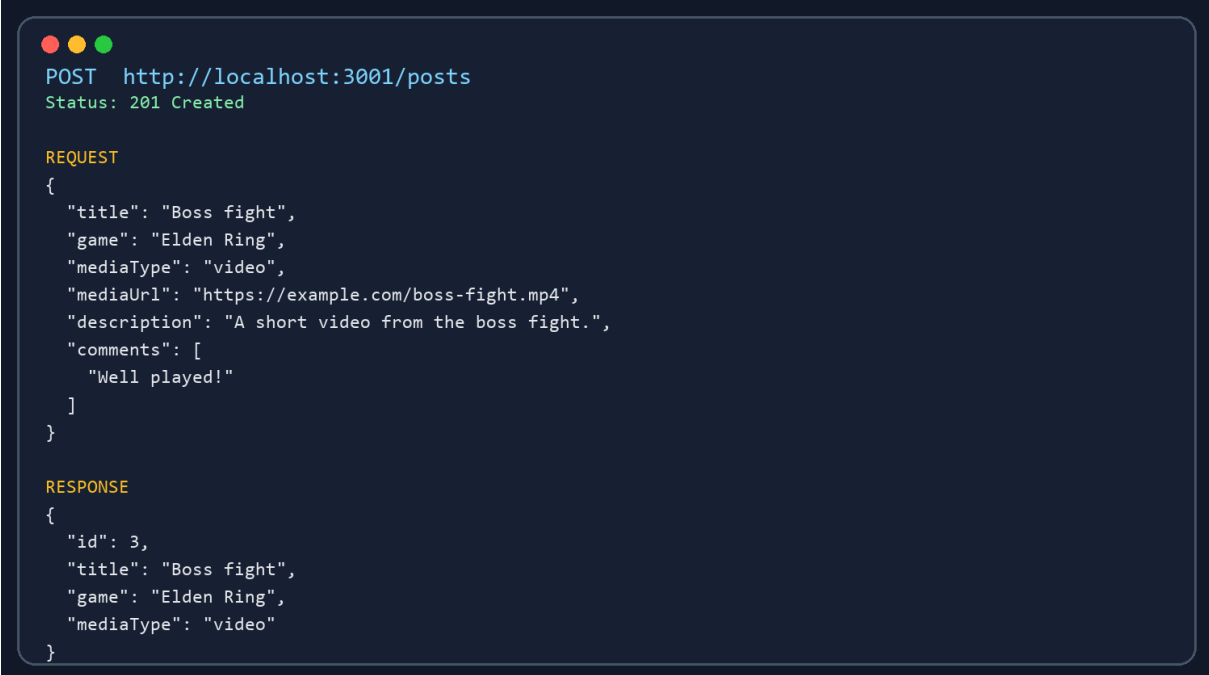


```
GET http://localhost:3001/posts/1
Status: 200 OK

REQUEST
Тело отсутствует

RESPONSE
{
  "id": 1,
  "title": "Sunset in Night City",
  "game": "Cyberpunk 2077",
  "mediaType": "screenshot",
  "mediaUrl": "https://example.com/night-city.jpg",
  "description": "A screenshot taken during an evening walk.",
  "comments": [
    "Great lighting!"
  ]
}
```

Рисунок 2 - Получение публикации по ID



```
● ● ●
POST http://localhost:3001/posts
Status: 201 Created

REQUEST
{
  "title": "Boss fight",
  "game": "Elden Ring",
  "mediaType": "video",
  "mediaUrl": "https://example.com/boss-fight.mp4",
  "description": "A short video from the boss fight.",
  "comments": [
    "Well played!"
  ]
}

RESPONSE
{
  "id": 3,
  "title": "Boss fight",
  "game": "Elden Ring",
  "mediaType": "video"
}
```

Рисунок 3 - Создание публикации



```
PUT http://localhost:3001/posts/3
Status: 200 OK

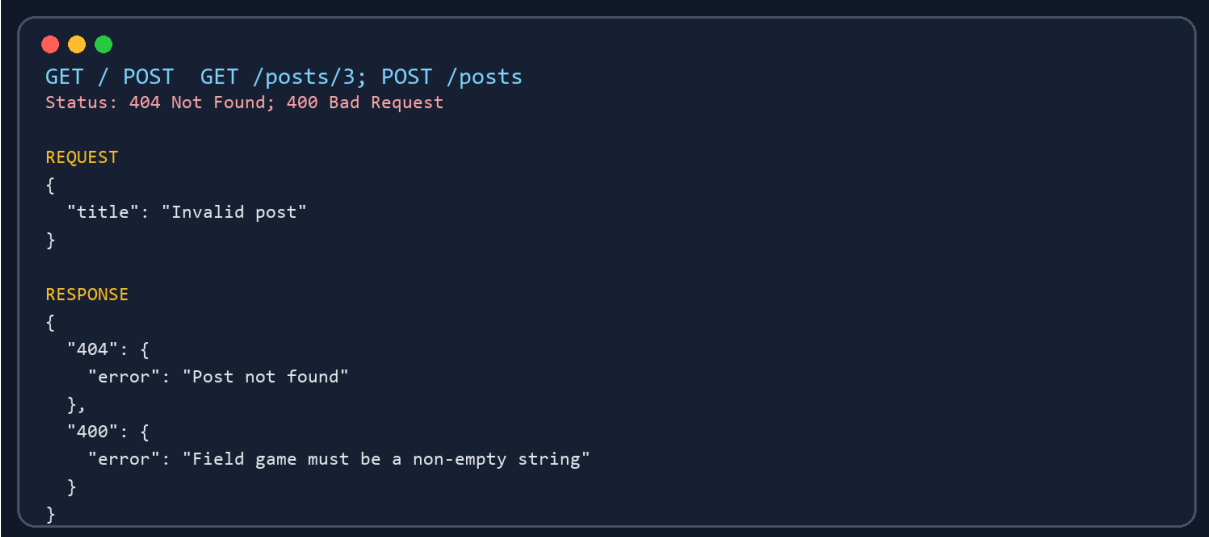
REQUEST
{
  "title": "Boss fight - no damage",
  "game": "Elden Ring",
  "mediaType": "video",
  "mediaUrl": "https://example.com/boss-fight-no-damage.mp4",
  "description": "The boss was defeated without taking damage.",
  "comments": [
    "Impressive!"
  ]
}

RESPONSE
{
  "id": 3,
  "title": "Boss fight - no damage",
  "game": "Elden Ring",
  "mediaType": "video"
}
```

Рисунок 4 - Полное обновление публикации



Рисунок 5 - Удаление публикации



```
GET / POST GET /posts/3; POST /posts
Status: 404 Not Found; 400 Bad Request

REQUEST
{
  "title": "Invalid post"
}

RESPONSE
{
  "404": {
    "error": "Post not found"
  },
  "400": {
    "error": "Field game must be a non-empty string"
  }
}
```

Рисунок 6 - Проверка ошибок 404 и 400

7 Ответы на контрольные вопросы

1. Какие методы HTTP реализованы?

GET - чтение, POST - создание, PUT - полное обновление, DELETE - удаление.

2. Как получить route params и query string?

Через req.params и req.query соответственно.

3. Для чего express.json()?

Middleware разбирает JSON из тела запроса и записывает объект в req.body.

4. Статусы создания и удаления?

201 Created и 204 No Content.

5. Что такое REST API?

Это интерфейс работы с ресурсами через HTTP. Основные принципы: клиент-серверное разделение, отсутствие состояния между запросами, единообразный интерфейс, кэширование и слоистая архитектура.

6. Как организовать глобальную обработку ошибок?

Добавить в конце middleware с параметрами (err, req, res, next).

7. Какая структура ошибки выбрана?

JSON-объект вида { error: "message" }.

8. Что вернет повторный DELETE или GET после удаления?

Статус 404 Not Found.

9. Зачем нужен nodemon?

Он автоматически перезапускает сервер после изменения файлов.

10. Разница PUT и PATCH?

PUT полностью заменяет данные ресурса, PATCH изменяет отдельные поля. Реализован PUT по заданию.

11. Как добавить CORS?

Теоретически: установить пакет cors и подключить app.use(cors()). В этой работе он не добавлялся.

12. Почему массив, а не БД?

Массив достаточен для изучения маршрутов. База данных будет подключаться в следующих работах для постоянного хранения.

13. Какой Content-Type нужен?

application/json.

14. Пример curl для POST.

```
curl -X POST http://localhost:3001/posts -H "Content-Type: application/json" -d '{"title":"Boss fight","game":"Elden Ring","mediaType":"video","mediaUrl":"https://example.com/video.mp4","description":"Boss battle"}'
```

15. Как проверять сервер после изменений?

Запускать npm run dev и повторять запросы из Postman или requests.http, проверяя статус и JSON.

16. Как запустить на другом компьютере?

Установить Node.js, клонировать репозиторий, перейти в Lab1, выполнить npm install и npm start.

8 Вывод

Создано серверное приложение на Node.js и Express.js для управления игровыми публикациями. Реализованы CRUD-маршруты, проверка данных и обработка ошибок. Проект разделен на routes, controllers и models, при этом данные по требованиям первой работы продолжают храниться в обычном массиве. База данных, JWT и интеграция с клиентской частью не реализовывались и оставлены для следующих лабораторных работ.